

# Spec-Driven Development

## Lección 0 – Introducción al curso

73.31 · ITBA · Construir software confiable con agentes de código.

# Qué vamos a ver hoy

**1 Presentaciones** – quiénes somos, quiénes son

**2 El problema** – por qué "anda" no alcanza

**3 Qué es SDD** – la idea central de la materia

**4 Las cuatro clases** – el arco completo y qué se entrega

**5 Cómo se evalúa** – entregas, presentación del equipo, sorteo

**6 Setup del ambiente** – y verificar que funciona

# Al final de esta clase podés...

**Explicar** qué problema resuelve Spec-Driven Development

**Ubicar** las cuatro clases y qué se entrega en cada una

**Saber** cómo se te evalúa y qué se espera de vos con un agente

**Tener** el ambiente andando y verificado

# Quiénes somos



**Fede Botti**

 [linkedin.com/in/fjbotti](https://linkedin.com/in/fjbotti)



**Fede Castañeda**

 [linkedin.com/in/federicocastaneda](https://linkedin.com/in/federicocastaneda)

# El problema

Los agentes escribieron el código. El problema pasó a ser otro.

# Lo que cambió

Escribir código dejó de ser el cuello de botella. Un agente produce en minutos lo que antes llevaba una tarde.

## Lo que se abarató

Tipear. Recordar sintaxis. Andamiaje. Traducir entre lenguajes.  
Escribir tests obvios.

## Lo que no

Decidir *qué* construir. Acordar qué significa "listo". Saber si lo entregado es lo que hacía falta.

La habilidad que escasea ya no es producir código – es **dirigir** a algo que produce código muy rápido.

# "Anda" no es lo mismo que "es lo que pedimos"

## ✅ Anda

Compila. No tira excepciones. La demo sale bien. Responde *¿corrió?*

## 🎯 Cumple el requerimiento

Hace lo que alguien necesitaba, y se puede *demostrar*. Responde *¿era esto?*

Un agente es excelente para lo primero. Lo segundo depende enteramente de **qué le pediste** y de **cómo lo verificás**.

Toda la materia vive en la brecha entre esas dos columnas.

# Qué es Spec-Driven Development

Sacar la fuente de verdad de la conversación



# La idea, en una slide

Sin una spec, hay tres cosas que **no se pueden hacer**:

## Definir el qué

Lo que no está acordado, el agente lo rellena con una suposición plausible. Nadie decidió; alguien asumió.

## Verificar

Sin criterios escritos *antes*, "listo" es una opinión. No hay contra qué contrastar lo que salió.

## Iterar eficiente

Reexplicás el contexto en cada sesión, revisás a mano cada paso, y rehacés lo que se entendió mal.

↓ y cada iteración sin verificar se apoya en la anterior sin verificar ↓

 Termina en un **castillo de arena**: corre, demuestra bien, y se cae al primer cambio real.

**SDD corta la cadena en el primer eslabón:** requerimientos + criterios de verificación, *antes* de implementar.

**Alcance explícito** – lo que queda afuera también está escrito.

**Un criterio de verificación por requerimiento** – "listo" pasa a ser chequeable.

**El contrato persiste** – no lo reexplicás, y podés delegar.

# Por qué ahora, y no hace diez años

Escribir requerimientos precisos siempre fue buena idea. Lo que cambió es el retorno.



## La ejecución es barata

Si implementar cuesta poco, especificar bien pasa a ser la parte cara – y la que decide el resultado.



## El agente lee la spec

Ya no es un documento que nadie abre: es el input que el ejecutor consume directamente.



## El error escala

Una ambigüedad ya no produce un bug: produce mil líneas coherentes en la dirección equivocada.

# Las cuatro clases

De una spec suelta a un agente conectado a todo tu ecosistema

# El arco de la materia



Cada clase usa lo de la anterior. No hay unidades sueltas.

# El mapa completo

| 1 · Greenfield              | 2 · Brownfield                 | 3 · Skills y agentes            | 4 · MCP y plugins             |
|-----------------------------|--------------------------------|---------------------------------|-------------------------------|
| El loop del agente          | Descubrir antes de especificar | Skills con trigger              | N+M en vez de N×M             |
| Base context                | Exploración vs. ejecución      | Rules siempre activas           | Resources · Prompts · Tools   |
| La spec es el contrato      | Superficie acotada             | Subagents: firewall de contexto | Seguridad por capas           |
| Requerimientos verificables | Invariantes y fuera de alcance | Hooks garantizan                | Economía de contexto          |
| Iterar, no big-bang         | Sin regresiones                |                                 | Un skill orquesta los servers |
|                             | Context capsule                |                                 |                               |

Cuatro clases, un solo hilo: sacar la **fuentes de verdad** de la conversación y ponerla donde el agente la lee.

# Qué entregás en cada clase

| Clase | Tema             | Entregable                                          | Modalidad |
|-------|------------------|-----------------------------------------------------|-----------|
| 1     | Greenfield       | Spec revisada + plan + iteración 1 andando          | En equipo |
| 2     | Brownfield       | Notas de exploración + spec acotada de un repo real | En equipo |
| 3     | Skills y agentes | Un skill + un subagent + un hook, con evidencia     | En equipo |
| 4     | MCP y plugins    | Un server MCP conectado y usado por un skill        | En equipo |

Las cuatro entregas son del **mismo equipo de trabajo**, en un **único repo** que va creciendo. **Pesan lo mismo**: 25% cada una, y la nota final es el promedio.

# Cómo se evalúa

Entregas, presentaciones por grupo, y qué tenés que poder defender

# Dos instancias



## La entrega

Un repo por equipo y el hash del commit a revisar, antes de la clase siguiente. **25% cada una**, promedio simple.



## La presentación de tu grupo

Cada grupo tiene **una clase asignada** y la abre presentando la tarea anterior en vivo. La clase entera la revisa.

La presentación no es examen sorpresa: es **práctica de revisión**. Leer un entregable y juzgar si cumple el contrato es la habilidad central de la materia – y se practica tanto exponiendo como preguntando.

El sorteo es en el momento. Por eso todos llegan listos: esa es la función.



# Cómo se entrega



## Un repo por equipo

El mismo las cuatro entregas. No es un repo por tarea: es un proyecto que crece.



## Link + hash del commit

Por mail a la cátedra. Se corrige **ese commit** y solo ese.



## Jueves al mediodía

De la **semana anterior** a la clase. No la noche anterior.

```
git log -1 --format=%H # el hash que mandás
```

El hash es tu forma de decir "**esto es lo que entrego**". Seguí trabajando en el repo sin miedo a que la entrega cambie abajo tuyo.

# Qué miramos al corregir

¿Cada requerimiento tiene un **criterio de verificación** observable?

¿El **alcance** está acotado y dice también qué queda afuera?

¿Las preguntas abiertas están **resueltas**, o quedaron "TBD"?

¿Lo que entregaste **corre**, y podés mostrarlo corriendo?

No se corrige el tamaño del feature. Se corrige la **disciplina**.

# Sobre usar agentes en las entregas

Se espera que uses un agente en todo. Esta materia es sobre eso.

## ✓ Lo esperado

Que el agente redacte, explore, implemente y verifique. Que vos decidas, revises y puedas justificar cada decisión.

## ⊘ El problema

Entregar algo que no leíste. En la presentación se nota en la primera pregunta, y ahí no hay agente que te ayude.

La regla: **todo lo que entregás lo tenés que poder defender** – por qué esa decisión, por qué ese alcance, por qué eso quedó afuera.

# Setup del ambiente

Abrí la máquina – esto lo hacemos ahora

# Lo que necesitás



## Un agente de código

Claude Code, Cursor, OpenCode, Codex CLI o equivalente – incluso con un **modelo open source local** (Ollama, LM Studio). Tiene que poder **leer y escribir archivos** y **correr comandos** – un chat solo no alcanza.



## El entorno de siempre

Git, un runtime para el lenguaje que elijas, y una terminal. La Lección 4 suma Docker.

Las clases muestran ejemplos con una herramienta concreta, pero los conceptos – skills, rules, hooks, MCP – existen en todas. Usá la que prefieras: nada de lo que enseñamos depende del modelo que haya del otro lado.

# Verificá que anda – ahora

Tres chequeos. Si los tres pasan, llegás listo a la Lección 1.

## 1 • El agente lee tu repo

Listame los archivos de este proyecto y decime qué hace.

## 2 • El agente escribe y ejecuta

Creá un archivo hola con un test que falle, corré el test y arreglalo.

## 3 • Cierra su propio loop

Que corra el test, vea que falla, lo corrija y lo vuelva a correr – sin que se lo pidas paso por paso.

# Dónde está el material



Todo está en el **campus virtual**, en la **carpeta de cada clase**.



Las **slides** de la clase, subidas después de darla



El **ejemplo guiado** – un caso resuelto de punta a punta, con su recorrido



La **consigna** de la tarea, con lo que hay que entregar

Una carpeta por clase. No hay material dando vueltas en otro lado.

# ¿Preguntas?

Próxima: Lección 1 – Agentes de código y SDD greenfield

Traé el ambiente andando. Los pares se asignan al inicio de la clase.